Pertemuan 1 & 2: Pengenalan Dart dan Dasar Pemrograman

---

1. Identitas Pembelajaran

Keterangan	Detail
Mata Kuliah	Pemrograman Berorientasi Objek (PBO)
SKS	3 SKS (1 Teori, 2 Praktik)
Semester	3
Pertemuan	1 & 2

---

2. Tujuan Pembelajaran

Setelah mengikuti pertemuan ini, mahasiswa diharapkan mampu:

1. Memahami latar belakang dan kegunaan bahasa pemrograman Dart.
2. Menginstal *Dart SDK* dan menyiapkan alat kerja (*Text Editor*).
3. Membuat proyek Dart pertama dan menjalankannya dari terminal.
4. Mendeklarasikan variabel dan memahami tipe-tipe data utama di Dart.
5. Menggunakan operator aritmatika, perbandingan, penugasan, dan logika.
6. Menerapkan struktur percabangan (`if-else`, `switch-case`) dan perulangan (`for`, `while`).
7. Memanfaatkan tipe data koleksi (`List`, `Set`, `Map`) untuk menyimpan banyak data.
8. Membuat dan memanggil *function* (fungsi).

---

3. Materi Pertemuan 1

A. Pengenalan Dart

Dart adalah bahasa pemrograman yang pertama kali diperkenalkan oleh Google pada tahun 2011. Dart dibuat oleh Lars Bak dan Kasper Lund.

Karakteristik utama Dart:

- Dart dirancang untuk pengembangan aplikasi *Frontend* (web dan *mobile*), namun juga dapat digunakan untuk *Backend*.
- Dart bersifat *Open Source* dan dapat digunakan secara bebas.
- Dart sangat populer karena menjadi bahasa utama framework Flutter untuk membuat aplikasi Android dan iOS.
- Dokumentasi resmi: `https://dart.dev/`

B. Dart SDK (*Software Development Kit*)

Dart merupakan bahasa yang bersifat *multi-platform*, artinya dapat diinstal di berbagai sistem operasi (Windows, Linux, macOS).

Untuk membuat aplikasi Dart, kita membutuhkan **Dart SDK**. SDK ini bertugas:

- Mengompilasi kode program Dart yang kita buat.
- Menjalankan aplikasi Dart.

C. Instalasi Dart SDK

Mengunduh: Kunjungi `https://dart.dev/get-dart` dan unduh paket *installer* sesuai sistem operasi.

Langkah Instalasi di Windows (Metode Manual ZIP):

1. Unduh file `.zip` dari halaman resmi dart.dev.
2. Ekstrak ke folder yang sederhana, misal: `C:\dart-sdk`.
3. Buka menu **Start**, ketik "*Edit the system environment variables*".
4. Klik **Environment Variables** → cari variabel **Path** pada bagian *System variables* → klik **Edit**.
5. Klik **New** dan masukkan: `C:\dart-sdk\bin`
6. Klik **OK** untuk menyimpan semua perubahan.
7. Buka terminal baru (*Command Prompt*) dan ketik: `dart --version`

Jika versi Dart muncul di terminal, instalasi berhasil.

D. *Text Editor* (Alat Penulisan Kode)

Untuk menulis kode Dart, kita membutuhkan *Text Editor*. Beberapa pilihan yang populer:

- **Visual Studio Code**: `https://code.visualstudio.com/` *(Paling direkomendasikan)*
- **IntelliJ IDEA**: `https://www.jetbrains.com/idea/`
- **Android Studio**: `https://developer.android.com/studio`

Setelah menginstal VS Code, instal ekstensi **Dart** dari *Marketplace* agar tersedia fitur pelengkap otomatis (*autocomplete*) dan deteksi kesalahan.

E. Membuat Proyek Dart (`dart create`)

Dart menyediakan perintah untuk membuat kerangka proyek secara otomatis.

Perintah di terminal:

```
dart create nama_proyek
```

Setelah dijalankan, akan terbentuk folder baru bernama `nama_proyek` dengan struktur sebagai berikut:

Folder/File	Fungsi
`bin/`	Tempat file utama program yang dieksekusi.
`lib/`	Tempat menyimpan kode logika atau *class* pendukung.

Folder/File	Fungsi
`pubspec.yaml`	File konfigurasi proyek, berisi nama proyek dan daftar paket yang digunakan.

F. Program "Hello World"

Dalam Dart, setiap file kode disimpan dengan ekstensi `.dart`. Setiap program Dart **wajib** memiliki sebuah fungsi utama bernama `main()`. Fungsi `main()` adalah titik awal (*entry point*) yang pertama kali dijalankan oleh Dart.

Membuat *Main Function*:

```
void main() {
  // Kode program ditulis di sini
}
```

Mencetak teks ke terminal menggunakan `print()`:

```
void main() {
  print('Hello World');
  print("Selamat datang di Dart!");
}
```

Output:

```
Hello World
Selamat datang di Dart!
```

Menjalankan kode program dari terminal:

```
dart run bin/nama_proyek.dart
```

---

4. Materi Pertemuan 2

A. Variabel

Variabel adalah tempat untuk menyimpan data. Variabel sangat berguna ketika kita ingin menggunakan data yang sama berkali-kali — cukup simpan sekali, panggil berkali-kali lewat nama variabelnya.

Format pendeklarasian variabel:

```
TipeData namaVariabel;
namaVariabel = nilainya;
```

Atau langsung sekaligus dalam satu baris:

```
TipeData namaVariabel = nilainya;
```

Penamaan variabel di Dart menggunakan gaya *camelCase*, contoh: `firstName`, `nilaiAkhir`, `hargaBarang`.

Contoh — Tanpa Variabel (Kurang Efisien):

```
void main() {
  print('Eko Kurniawan Khannedy');
  print('Eko Kurniawan Khannedy');
  print('Eko Kurniawan Khannedy');
}
```

Contoh — Menggunakan Variabel (Lebih Efisien):

```
void main() {
  String nama = 'Eko Kurniawan Khannedy';
  print(nama);
  print(nama);
  print(nama);
}
```

Kata Kunci `var`: Kita bisa menggunakan `var` sebagai pengganti tipe data. Dart akan otomatis mendeteksi tipe datanya dari nilai yang kita berikan.

```
void main() {
  var nama = 'Budi';   // Dart mendeteksi ini sebagai String
  var umur = 20;       // Dart mendeteksi ini sebagai int
  print(nama);
  print(umur);
}
```

Kata Kunci `final`: Variabel `final` tidak dapat diisi ulang setelah pertama kali diberi nilai. Nilainya dibaca saat program berjalan (*runtime*).

```
void main() {
  final String kota = 'Jakarta';
  // kota = 'Surabaya'; // ERROR: variabel final tidak bisa diubah
```

```
  print(kota);
}
```

Kata Kunci `const`: Variabel `const` bersifat konstan sepenuhnya. Nilainya sudah ditetapkan sejak tahap kompilasi (*compile-time*), bukan saat program berjalan.

```
void main() {
  const double phi = 3.14159;
  print(phi);
}
```

Kata Kunci `late`: Variabel `late` memungkinkan kita mendeklarasikan variabel tanpa langsung mengisinya. Nilainya baru akan diisi nanti saat variabel tersebut pertama kali diakses.

```
late String pesan;
// ... beberapa kode lain ...
pesan = 'Halo!';
print(pesan);
```

B. Komentar

Komentar adalah teks dalam kode yang **tidak akan dieksekusi** oleh program. Komentar berguna untuk memberikan penjelasan atau catatan pada kode.

Jenis-jenis komentar di Dart:

```
// Ini adalah komentar satu baris

/*
  Ini adalah komentar
  lebih dari satu baris
*/

/// Ini adalah komentar dokumentasi (biasa digunakan untuk menjelaskan
fungsi/class)
```

C. Tipe Data `Number` (Angka)

Dart memiliki dua tipe data untuk angka:

- `int`: Bilangan bulat (tanpa koma). Contoh: `10`, `-5`, `1000`.
- `double`: Bilangan desimal (dengan koma). Contoh: `3.14`, `-0.5`. Penulisan pemisah desimal menggunakan **titik** (`.`), bukan koma.

Jika kita ingin satu variabel bisa menampung baik `int` maupun `double`, gunakan tipe `num`.

```
void main() {
  int jumlah = 10;
  double harga = 5500.50;
  num nilai = 90;  // bisa diubah ke int atau double

  print(jumlah);
  print(harga);
  print(nilai);
}
```

Output:

```
10
5500.5
90
```

D. Tipe Data `Boolean`

`bool` adalah tipe data yang hanya memiliki dua kemungkinan nilai: **`true`** (benar) atau **`false`** (salah).

```
void main() {
  bool lulus = true;
  bool aktif = false;

  print(lulus);   // Output: true
  print(aktif);   // Output: false
}
```

E. Tipe Data `String` (Teks)

`String` adalah tipe data untuk menyimpan teks atau karakter. Nilai String diapit oleh tanda kutip satu (`'...'`) atau kutip dua (`"..."`).

Contoh dasar:

```
void main() {
  String nama = 'Budi Santoso';
  String jurusan = "Ilmu Komputer";
  print(nama);
  print(jurusan);
}
```

String Interpolation: Kita bisa menyisipkan nilai variabel langsung ke dalam String menggunakan tanda `$`.

```
void main() {
  String nama = 'Eko';
  int umur = 30;
  print('Nama saya $nama dan umur saya $umur tahun.');
  // Untuk ekspresi lebih kompleks, gunakan ${...}
  print('Dua ditambah tiga adalah ${2 + 3}');
}
```

Output:

```
Nama saya Eko dan umur saya 30 tahun.
Dua ditambah tiga adalah 5
```

Karakter *Backslash* (`\`): Digunakan untuk menulis karakter khusus di dalam String.

- `\$` → Menulis karakter dolar `$` secara harfiah.
- `\'` → Menulis karakter kutip satu di dalam String berkutip satu.
- `\n` → Membuat baris baru.

```
void main() {
  print('Harga barang: \$100');        // Output: Harga barang: $100
  print('Baris pertama\nBaris kedua'); // Pindah ke baris baru
}
```

Menggabungkan String:

```
void main() {
  String depan = 'Halo';
  String belakang = 'Dunia';
  print(depan + ' ' + belakang); // Output: Halo Dunia
}
```

F. Tipe Data `dynamic`

Tipe `dynamic` memungkinkan sebuah variabel menampung data dari tipe apa pun, dan bisa berubah-ubah kapan saja.

```
void main() {
  dynamic data = 'Ini String';
  print(data);  // Output: Ini String
```

```
  data = 100;   // Kini berubah menjadi int
  print(data);  // Output: 100

  data = true;  // Kini berubah menjadi bool
  print(data);  // Output: true
}
```

G. Konversi Tipe Data

Kadang kita perlu mengubah satu tipe data ke tipe data lain. Misalnya, input dari pengguna biasanya berupa `String`, tetapi kita perlu memprosesnya sebagai angka.

Konversi String ↔ Number:

```
void main() {
  // String ke int
  int angka = int.parse('100');
  print(angka + 5); // Output: 105

  // String ke double
  double desimal = double.parse('3.14');
  print(desimal);   // Output: 3.14

  // int ke String
  String teks = angka.toString();
  print(teks);      // Output: 100

  // double ke int (membuang bagian desimal)
  int bulat = desimal.toInt();
  print(bulat);     // Output: 3
}
```

Konversi Boolean → String:

```
void main() {
  bool status = true;
  String teks = status.toString();
  print(teks); // Output: true
}
```

H. Operator

H.1. Operator Aritmatika

Digunakan untuk melakukan operasi matematika.

Operator	Keterangan	Contoh	Hasil
`+`	Penjumlahan	`10 + 3`	`13`
`-`	Pengurangan	`10 - 3`	`7`
`*`	Perkalian	`10 * 3`	`30`
`/`	Pembagian (hasil `double`)	`10 / 3`	`3.333...`
`~/`	Pembagian bulat (hasil `int`)	`10 ~/ 3`	`3`
`%`	Sisa bagi (*Modulo*)	`10 % 3`	`1`

```
void main() {
  int a = 10;
  int b = 3;

  print(a + b);  // Output: 13
  print(a - b);  // Output: 7
  print(a * b);  // Output: 30
  print(a / b);  // Output: 3.3333333333333335
  print(a ~/ b); // Output: 3
  print(a % b);  // Output: 1
}
```

H.2. Operator Perbandingan

Digunakan untuk membandingkan dua nilai. Hasil dari operator ini selalu berupa `bool` (`true` atau `false`).

Operator	Keterangan	Contoh	Hasil
`==`	Sama dengan	`5 == 5`	`true`
`!=`	Tidak sama dengan	`5 != 3`	`true`
`>`	Lebih dari	`5 > 3`	`true`
`<`	Kurang dari	`5 < 3`	`false`
`>=`	Lebih dari atau sama dengan	`5 >= 5`	`true`
`<=`	Kurang dari atau sama dengan	`5 <= 3`	`false`

```
void main() {
  int nilai = 75;

  print(nilai == 75);  // Output: true
  print(nilai > 80);   // Output: false
  print(nilai >= 75);  // Output: true
}
```

H.3. Operator Penugasan (*Assignment*)

Digunakan untuk menetapkan atau memperbarui nilai suatu variabel.

Bentuk Panjang	Bentuk Singkat
`a = a + 10`	`a += 10`
`a = a - 10`	`a -= 10`
`a = a * 10`	`a *= 10`
`a = a / 10`	`a /= 10`
`a = a % 10`	`a %= 10`

Increment dan *Decrement*:

- `a++` atau `++a` → menambah nilai `a` sebesar 1.
- `a--` atau `--a` → mengurangi nilai `a` sebesar 1.

```
void main() {
  int stok = 100;
  stok -= 10;  // Sama dengan: stok = stok - 10
  print(stok); // Output: 90

  stok++;      // Sama dengan: stok = stok + 1
  print(stok); // Output: 91
}
```

H.4. Operator Logika

Digunakan untuk menggabungkan dua atau lebih ekspresi `bool`.

Operator	Nama	Keterangan
`&&`	AND (Dan)	Menghasilkan `true` jika **kedua** sisi bernilai `true`.
`||`	OR (Atau)	Menghasilkan `true` jika **salah satu** sisi bernilai `true`.
`!`	NOT (Kebalikan)	Membalik nilai `true` menjadi `false`, atau sebaliknya.

Tabel kebenaran `&&`:

Nilai 1	Operator	Nilai 2	Hasil
`true`	`&&`	`true`	`true`
`true`	`&&`	`false`	`false`
`false`	`&&`	`true`	`false`
`false`	`&&`	`false`	`false`

Tabel kebenaran `||`:

Nilai 1	Operator	Nilai 2	Hasil
`true`	`||`	`true`	`true`
`true`	`||`	`false`	`true`
`false`	`||`	`true`	`true`
`false`	`||`	`false`	`false`

```
void main() {
  bool sudahLogin = true;
  bool punyaIzin = false;

  // AND: keduanya harus true
  print(sudahLogin && punyaIzin); // Output: false

  // OR: cukup salah satu true
  print(sudahLogin || punyaIzin); // Output: true

  // NOT: kebalikan
  print(!sudahLogin); // Output: false
}
```

I. Struktur Percabangan (*Branching*)

Percabangan digunakan agar program dapat mengambil keputusan berdasarkan kondisi tertentu.

I.1. Pernyataan `if` dan `else`

Blok `if` dijalankan jika kondisi bernilai `true`. Blok `else` dijalankan jika kondisi bernilai `false`.

```
void main() {
  int nilai = 65;

  if (nilai >= 75) {
    print('Lulus');
  } else {
    print('Tidak Lulus');
  }
}
```

Output:

```
Tidak Lulus
```

I.2. Pernyataan `else if`

Digunakan ketika ada lebih dari dua kondisi yang harus diperiksa.

```
void main() {
  int nilai = 78;

  if (nilai >= 90) {
    print('Grade A');
  } else if (nilai >= 80) {
    print('Grade B');
  } else if (nilai >= 70) {
    print('Grade C');
  } else {
    print('Tidak Lulus');
  }
}
```

Output:

```
Grade C
```

I.3. Pernyataan `switch-case`

`switch` cocok digunakan ketika kita hanya perlu memeriksa kesamaan nilai (`==`) terhadap satu variabel. Pernyataan `break` wajib ditulis di akhir setiap `case` untuk menghentikan eksekusi.

```
void main() {
  String hari = 'Senin';

  switch (hari) {
    case 'Senin':
      print('Mulai minggu kerja');
      break;
    case 'Jumat':
      print('Akhir minggu kerja');
      break;
    case 'Sabtu':
    case 'Minggu':
      print('Hari libur');
      break;
    default:
      print('Hari kerja biasa');
  }
}
```

Output:

```
Mulai minggu kerja
```

I.4. *Null* dan *Ternary Operator*

`Null` mewakili kondisi tidak ada data / data kosong. Secara bawaan, variabel di Dart tidak boleh bernilai `null`. Untuk mengizinkan `null`, tambahkan tanda `?` setelah tipe data.

```dart
void main() {
  String? nama = null; // Variabel yang boleh null
  print(nama);         // Output: null
}
```

Ternary Operator adalah cara singkat pengganti `if-else` sederhana: `kondisi ? nilaiJikaTrue : nilaiJikaFalse`

```dart
void main() {
  int nilai = 80;
  String hasil = nilai >= 75 ? 'Lulus' : 'Tidak Lulus';
  print(hasil); // Output: Lulus
}
```

J. *Null Safety*

NullPointerException (error yang terjadi saat mengakses data yang ternyata bernilai `null`) adalah salah satu kesalahan paling umum di banyak bahasa pemrograman. Dart memiliki fitur ***Null Safety*** untuk mencegah hal ini.

Aturan utama:

- Secara default, variabel **tidak boleh** bernilai `null`.
- Jika variabel **boleh** bernilai `null`, tambahkan `?` pada tipe datanya.

Null Check (memeriksa null sebelum mengakses):

```dart
void main() {
  String? nama = null;

  if (nama != null) {
    print(nama.length); // Aman, hanya diakses jika tidak null
  }
}
```

Default Value dengan Operator `??`: Operator `??` memberikan nilai pengganti jika variabel bernilai `null`.

```
void main() {
  String? username = null;
  String tampil = username ?? 'Tamu'; // Jika null, gunakan 'Tamu'
  print(tampil); // Output: Tamu
}
```

Konversi Paksa dengan `!`: Operator `!` memaksa konversi dari tipe *nullable* ke *non-nullable*. Gunakan ini hanya jika Anda **yakin** datanya tidak `null`, karena jika ternyata `null`, program akan error.

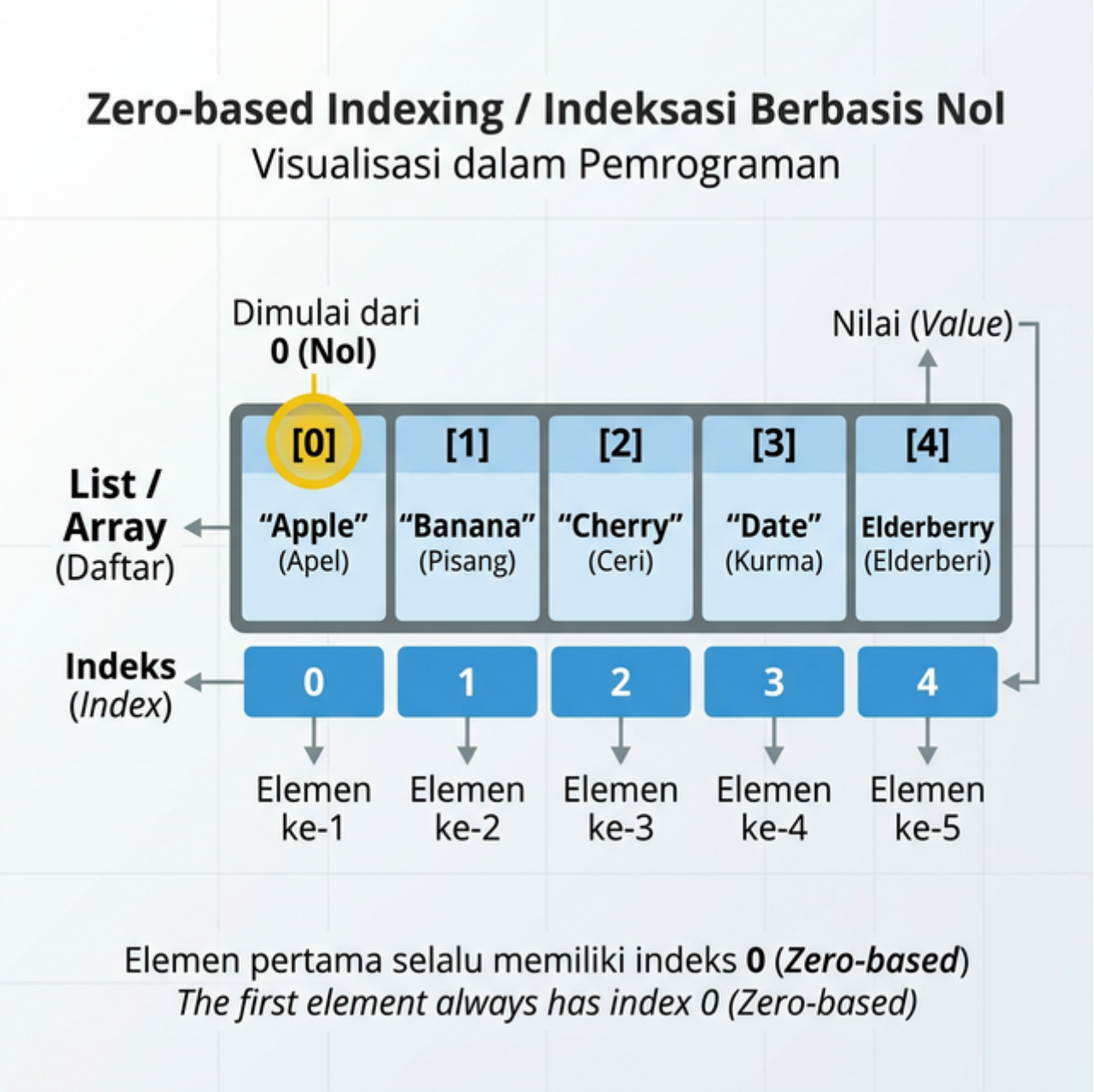
```
void main() {
  String? kode = '12345';
  String pasti = kode!; // Paksa konversi
  print(pasti);         // Output: 12345
}
```

K. Tipe Data Koleksi

Koleksi digunakan untuk menyimpan **banyak data** dalam satu variabel.

K.1. `List` (Daftar Berurutan)

`List` menyimpan data secara berurutan. Setiap elemen memiliki **indeks** yang dimulai dari **0**.




```dart
void main() {
  // Membuat List
  List<String> buah = ['Apel', 'Jeruk', 'Mangga'];

  // Mengakses data berdasarkan indeks
  print(buah[0]); // Output: Apel
  print(buah[1]); // Output: Jeruk

  // Menambah data ke akhir List
  buah.add('Anggur');
  print(buah.length); // Output: 4

  // Mengubah data pada indeks tertentu
  buah[0] = 'Pisang';
  print(buah[0]); // Output: Pisang

  // Menghapus data berdasarkan indeks
  buah.removeAt(1);
```

```
  print(buah); // Output: [Pisang, Mangga, Anggur]
}
```

Deklarasi `List` langsung dengan data:

```
var angka = [1, 2, 3, 4, 5];
var nama = <String>['Eko', 'Budi', 'Joko'];
```

K.2. `Set` (Himpunan Tanpa Duplikat)

`Set` hampir sama dengan `List`, dengan dua perbedaan utama:

1. **Tidak menerima data duplikat** — jika ada yang sama, akan diabaikan.
2. **Tidak memiliki indeks** — urutan data tidak dijamin.

```
void main() {
  Set<String> kota = {'Jakarta', 'Bandung', 'Surabaya'};
  kota.add('Medan');
  kota.add('Jakarta'); // Duplikat, akan diabaikan

  print(kota);        // Output: {Jakarta, Bandung, Surabaya, Medan}
  print(kota.length); // Output: 4
}
```

K.3. `Map` (Pasangan Kunci-Nilai)

`Map` menyimpan data dalam bentuk pasangan **kunci** (*key*) dan **nilai** (*value*). Berbeda dengan `List` yang menggunakan indeks angka, `Map` menggunakan kunci yang bisa kita tentukan sendiri.

```
void main() {
  // Format: Map<TipeKunci, TipeNilai>
  Map<String, int> hargaMenu = {
    'Nasi Goreng': 15000,
    'Mie Ayam': 12000,
    'Es Teh': 5000,
  };

  // Mengambil data berdasarkan kunci
  print(hargaMenu['Nasi Goreng']); // Output: 15000

  // Menambah atau mengubah data
  hargaMenu['Jus Alpukat'] = 18000;
  hargaMenu['Es Teh'] = 6000; // Mengubah nilai yang sudah ada

  // Menghapus data
  hargaMenu.remove('Mie Ayam');
```

```
  print(hargaMenu.length); // Output: 3
}
```

L. Perulangan (*Looping*)

Perulangan digunakan untuk menjalankan blok kode secara berulang.

L.1. `for` Loop

Digunakan ketika jumlah perulangan sudah diketahui.

Struktur:

```
for (inisialisasi; kondisi; langkah) {
  // blok kode
}
```

```
void main() {
  for (int i = 1; i <= 5; i++) {
    print('Perulangan ke-$i');
  }
}
```

Output:

```
Perulangan ke-1
Perulangan ke-2
Perulangan ke-3
Perulangan ke-4
Perulangan ke-5
```

L.2. `while` Loop

Digunakan ketika jumlah perulangan tidak diketahui di awal. Perulangan terus berjalan selama kondisi bernilai `true`.

```
void main() {
  int hitungan = 1;

  while (hitungan <= 3) {
    print('Hitung: $hitungan');
    hitungan++; // Penting: tanpa ini, akan terjadi perulangan tanpa henti
```

```
  }
}
```

Output:

```
Hitung: 1
Hitung: 2
Hitung: 3
```

L.3. `do-while` Loop

Hampir sama dengan `while`, perbedaannya: blok kode dijalankan **minimal sekali** meskipun kondisi langsung bernilai `false`, karena pengecekan kondisi dilakukan **setelah** blok kode.

```
void main() {
  int x = 10;

  do {
    print('Nilai x: $x'); // Tetap dijalankan sekali walaupun kondisi false
    x++;
  } while (x < 5); // Kondisi ini false sejak awal
}
```

Output:

```
Nilai x: 10
```

L.4. `break` dan `continue`

- `break`: Menghentikan seluruh perulangan secara langsung.
- `continue`: Melewati satu siklus perulangan, lalu lanjut ke siklus berikutnya.

```
void main() {
  // Contoh break
  for (int i = 1; i <= 10; i++) {
    if (i == 5) break; // Hentikan saat i = 5
    print(i);
  }
  // Output: 1 2 3 4

  print('---');

  // Contoh continue
  for (int i = 1; i <= 5; i++) {
```

```
    if (i == 3) continue; // Lewati angka 3
    print(i);
  }
  // Output: 1 2 4 5
}
```

L.5. `for-in` (Iterasi Koleksi)

`for-in` digunakan untuk mengakses setiap elemen dalam sebuah koleksi (`List`, `Set`) secara langsung, tanpa perlu mengelola indeks.

```
void main() {
  List<String> makanan = ['Rendang', 'Sate', 'Gado-Gado'];

  for (var item in makanan) {
    print(item);
  }
}
```

Output:

```
Rendang
Sate
Gado-Gado
```

M. Fungsi (*Function*)

Fungsi adalah blok kode yang diberi nama dan dapat dipanggil (digunakan) berkali-kali dari bagian mana pun dalam program.

M.1. Membuat dan Memanggil Fungsi

```
// Mendefinisikan fungsi
void sapa() {
  print('Halo! Selamat datang.');
}

void main() {
  sapa(); // Memanggil fungsi
  sapa(); // Bisa dipanggil berkali-kali
}
```

Output:

```
Halo! Selamat datang.
Halo! Selamat datang.
```

M.2. Fungsi dengan Parameter

Fungsi dapat menerima data dari luar melalui **parameter**.

```dart
// Fungsi dengan dua parameter
void tampilkanInfo(String nama, int umur) {
  print('Nama: $nama, Umur: $umur tahun');
}

void main() {
  tampilkanInfo('Budi', 22);
  tampilkanInfo('Sari', 19);
}
```

Output:

```
Nama: Budi, Umur: 22 tahun
Nama: Sari, Umur: 19 tahun
```

M.3. Fungsi dengan Nilai Kembalian (*Return Value*)

Fungsi bisa mengembalikan hasil perhitungan menggunakan kata kunci `return`.

```dart
// Fungsi mengembalikan nilai int
int tambah(int a, int b) {
  return a + b;
}

void main() {
  int hasil = tambah(5, 3);
  print('Hasil: $hasil'); // Output: Hasil: 8
}
```

M.4. *Optional Parameter* (Parameter Opsional)

Parameter dalam kurung siku `[]` bersifat opsional — boleh tidak diisi saat pemanggilan fungsi.

```dart
void salam(String nama, [String? gelar]) {
  if (gelar != null) {
    print('Halo, $gelar $nama');
```

```
  } else {
    print('Halo, $nama');
  }
}

void main() {
  salam('Budi');          // Output: Halo, Budi
  salam('Sari', 'Dr.');   // Output: Halo, Dr. Sari
}
```

M.5. *Named Parameter* (Parameter Bernama)

Parameter dalam kurung kurawal `{}` dapat dipanggil dengan menyebutkan nama parameternya saat fungsi dipanggil, sehingga urutannya tidak harus sesuai.

```
void biodata({required String nama, required int umur}) {
  print('Nama: $nama | Umur: $umur');
}

void main() {
  biodata(nama: 'Eko', umur: 30);
  biodata(umur: 25, nama: 'Joko'); // Urutan boleh dibalik
}
```

M.6. *Function Short Expression* (Fungsi Panah)

Jika isi fungsi hanya satu baris ekspresi, kita bisa menyederhanakannya dengan sintaks panah `=>`.

```
int kali(int a, int b) => a * b;

void main() {
  print(kali(4, 5)); // Output: 20
}
```

---

5. Tugas Praktikum Mandiri (Asinkronus — 180 Menit)

Studi Kasus: Simulasi Aplikasi Kasir Sederhana Berbasis Terminal

Tujuan: Mengintegrasikan pemahaman variabel, operator, input dari pengguna, logika percabangan, dan tipe data koleksi dalam satu program yang berjalan di terminal.

Spesifikasi Proyek:

1. **Buat proyek baru**: Jalankan perintah `dart create proyek_kasir_NIM` di terminal.
2. **Buat data menu**: Deklarasikan daftar menu makanan dan harganya menggunakan `Map<String, int>` yang berisi minimal 3 item menu.

3. **Input dari pengguna**: Gunakan pustaka `import 'dart:io';` dan `stdin.readLineSync()` untuk meminta pengguna memasukkan nama menu yang dipilih dan jumlah porsi yang dipesan.
4. **Hitung total harga**: Kalikan harga menu dengan jumlah yang dipesan. Tambahkan PPN 11% menggunakan operator perkalian (`* 0.11`).
5. **Logika diskon**: Gunakan `if-else` — jika total pembayaran >= Rp100.000, kurangi Rp10.000 sebagai diskon.
6. **Cetak struk**: Tampilkan ringkasan pesanan menggunakan `print()` dan *String interpolation* (`$`). Tambahkan garis pembatas seperti `==== STRUK PEMBAYARAN ====`.
7. **Penanganan null**: Gunakan operator `??` saat menerima input dari `stdin.readLineSync()` untuk menghindari error jika input kosong.
8. **Pengumpulan tugas**: Jalankan program dengan `dart run`, ambil *screenshot* terminal saat program berjalan. Kumpulkan file `.dart` dan *screenshot* ke LMS sebelum batas waktu yang ditentukan.